

Assignment 2

Arrays & Procedural routines

Design Specs:

The module implements a simple combinational processor that performs one of four operations on an 8-bit input data word based on a 2-bit operation select input.

Port Name	Port Type	Width	Description
data_in	Input	8 bits	Input data word to be processed.
op_sel	Input	2 bits	Operation select signal to choose the required operation.
data_out	Output	8 bits	Result of the selected operation applied to data_in.

Output changes immediately based on operation type and inputs according to the following table:

op_sel	Operation	Output Result
2'b00	Increment	data_out = data_in + 1
2'b01	Decrement	data_out = data_in - 1
2'b10	Bitwise NOT	data_out = ~data_in
2'b11	Left Shift	data_out = data_in << 1
default	Pass-through	data_out = data_in

Objective:

We need test the **multi_op_processor** combinational DUT using proper data types and routines,

- Declare **Dynamic array** of proper **datatype** (Choose the most appropriate one in your opinion and mention why you chose it) to store the stimulus for the design.
- **Queue** to Collect the output values from the design.
- **Associative array** holds the expected values from the golden model and storing them with index type **int** (the index represents the stimulus number).

- Declare the following tasks or functions with the following specs:

- ✦ **configure_stim_storage (int size):** This **function** will be used to configure the size of the dynamic array that holds the stimulus with the size *u* passed to the function. **And always clear the content of the actual data queue and expected data associative array after allocating a new size to guarantee the right data being stored in these storages with each re-configuration for the size.**
- **generate_stimulus (<dynamic array (as reference)>):** To randomize the stimulus and save it in the dynamic array storage.
- **drive_stim ():** Task which iterates through previously prepared stimulus with the dynamic array and send the stimulus to DUT.
- **golden_model (<Associative Array as reference>):** This task takes the stimulus you sent to design and calculate the expected results then store it in the associative array to be used afterward with checks.
- **collect_output_data (<data from design outputs>):** To collect the data have been read from the design and store it with the queue.
- **check_results (<Expected data>, <Actual data>):** Takes the expected data stored in the Associative array previously prepared in golden model **and** the actual output data from the design stored in the queue **and** iterates through them and makes the checks with proper display statements to indicates whether the data is successful as expected or not.
- **reconfigure_stim (<dynamic array>):** To just make **randomization** on the order of the stimulus stored previously in the dynamic array using the appropriate array sorting method (what will *u* choose?). **Note** after using this function you will need to update the expected values order and re calculate it again based on the new order.

Helpful Steps:

The expected flow you will walk through is to

- ✦ 1. Configure the stimulus storage size with **100 locations** for the stimulus.
- ✦ 2. Prepare the stimulus by generating them and store them (expected to be 100).
- ✦ 3. Driving the stimulus to the design.
- ✦ 4. Sending the stimulus generated to the golden model to calculate the expected data and store them.
- ✦ 5. Collecting the data from the design after some delay 1ns and gathering them into the meant storage for output data from the design as mentioned above.
- ✦ 6. Making the checks on the collected stored actual data from design comparing with the expected stored data from the golden model.
- ✦ 7. Repeat the above steps after re-configuring the dynamic array size with **200 locations**.
8. Then make some re-ordering on the stimulus storage you have and repeat the testing process again of re-calculating the golden model, driving the stimulus and making the checks again.

Use the tasks and functions described above to perform all the steps mentioned to automate the process

Deliverables:

U have to deliver

- **Zip Folder** contains:
 - The implemented **.sv** files and the **design files** and **run.do** file.
- **Separated PDF** contains:
 - ✍ Snippets for the implemented code.
 - ✍ Snippet for the waveform shows the values for created variables
 - ✍ Snippets for the logs show the all printed values.
 - ✍ Snippet for Do file you have used for automating the process.

The delivered zip file must be named like **your_name_Assignment_1.rar** for example:
Hassan_Khaled_Assignment_1.rar also the PDF file **Hassan_Khaled_Assignment_1.pdf**

Deadline:

The deadline is Thu 17/7 at 6 PM

Good Luck